

2026 首届 openvela AI 硬件开发者大赛 · 技术报告

作品名称：人物识别摄像头（RK3588 三域 AMP 实时识别终端） 仓库：
contest2026_160_xiangyonghusuoxiangdui

1. 信息表

项目	内容
作品名称	人物识别摄像头（openvela × Linux 三域 AMP 实时人物识别终端）
队伍名称	想用户所想队（contest2026_160_xiangyonghusuoxiangdui）
团队分工	陈景霖
选题方向	RK3588 新硬件平台适配

2. 摘要

本作品在**单颗 RK3588 SoC**上实现 openvela(NuttX) 与 Linux 的**三域 AMP 并行**：一颗 A55（cpu_l3）与一直闲置的**PMU Cortex-M0**各跑一个独立 NuttX 实例，其余 7 颗 A55 跑 Linux，三域共用一根调试串口，并通过 rpmsg + 共享内存协作，最终做成**实时人物识别摄像头**—— NuttX 直接编程 VOP 硬件图层出画面、跑 LVGL 与触摸，Linux 负责 imx415 采集与 RKNN NPU 推理。主体工作是为两个此前无支持的目标（RK3588 A55、PMU M0）新建完整 chip / board 层，并修复 NuttX 通用层 GICv3 redistributor 选择与 uart_rpmsg 两处缺陷。真机实测：相机页 **21.2 fps**、双路 ISP 满帧**零丢帧**（30.0 / 30.2 fps）、NPU 单核推理 **17.4 ms（≈55 fps）**、连续运行**零断言零应用层报错**、冷启动两次顺序一致。

3. 正文

3.1 绪论

3.1.1 项目背景与问题定义

AI 硬件产品长期面临一个两难：既想要 RTOS（实时操作系统）的**确定性与低启动延迟**，让 UI 不卡、开机即用、可控可裁剪；又离不开 Linux 那套庞大的**驱动生态与 AI 工具链**——摄像头 ISP、3A（自动曝光/白平衡/对焦）、NPU 推理（RKNN）在主流 SoC 上几乎只有 Linux 侧有闭源支持。

通常的做法只能二选一，或者干脆上两颗芯片（一颗 MCU + 一颗 AP），带来成本、功耗、同步复杂度的上升。

本作品面向**智能安防摄像头 / 交互式 AI 终端**这一典型场景，提出在**一颗 SoC 内部**用 AMP（非对称多处理）把两者合起来：让 openvela 负责界面与交互（确定性、崩溃隔离），Linux 负责重驱动与算力（生态、NPU），二者在片内高速协作。

3.1.2 技术难点

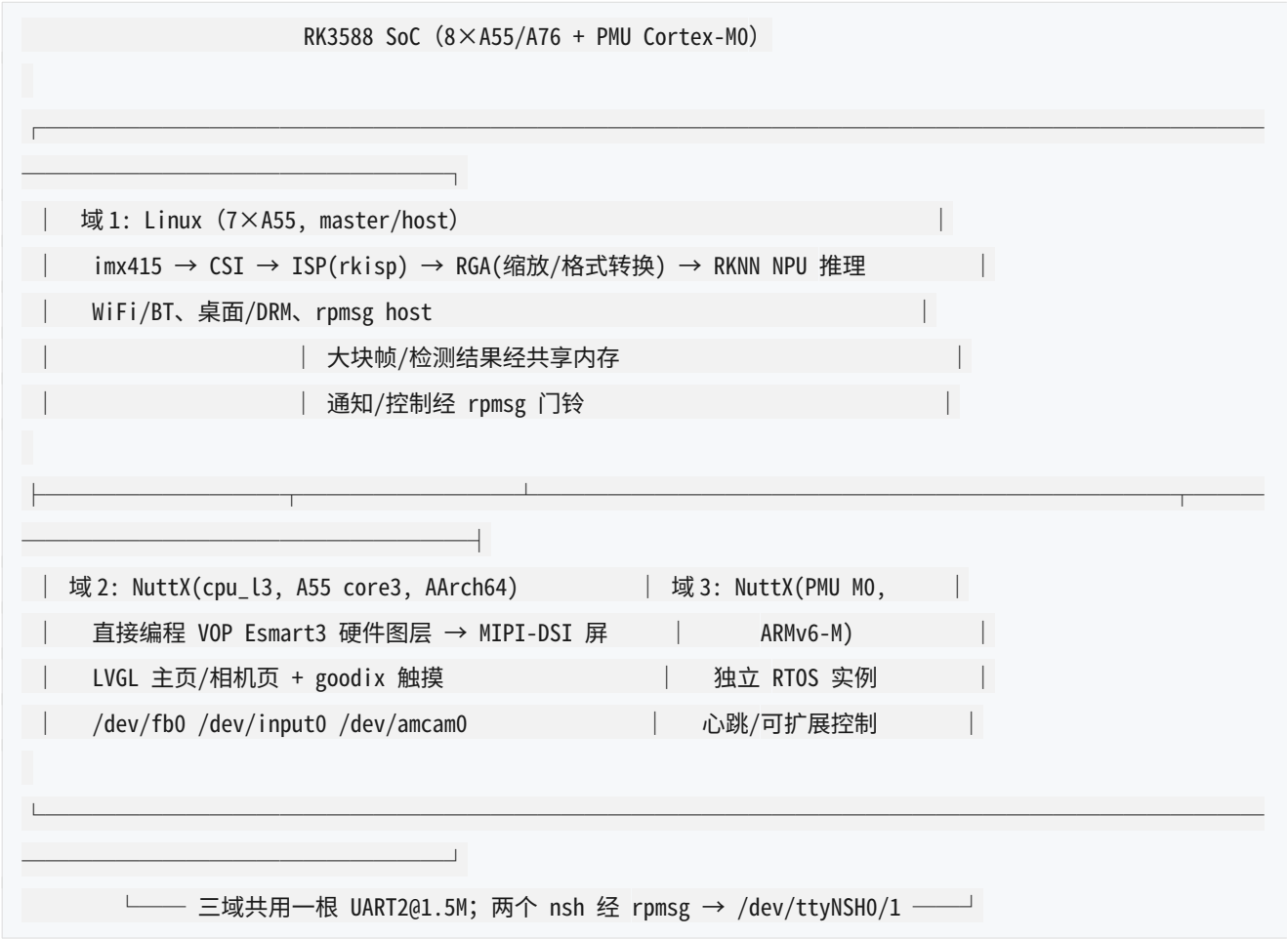
1. **两个此前不被 openvela 支持的目标的从零适配**：NuttX 上游只有 rk3399 的 arm64 chip 层，RK3588 的 A55 需新建整套 chip 层；而 **PMU Cortex-M0 官方 AMP 参考里完全没有**——没有 rpmsg link、没有示例固件，只占一块保留内存。
2. **三域并行下的中断与定时器共存**：NuttX 从核要在 Linux 已经初始化过的 GICv3 上正确拿到自己的 redistributor 与定时器 PPI，且 rpmsg 门铃中断要能在 Linux 反复重配 GIC 的情况下存活。
3. **显示不经过 Linux**：要从 Linux 的 VOP 手里安全地摘出一个硬件图层给 NuttX 直接扫描，且不破坏 Linux 自身的显示。
4. **跨域内存一致性**：AMP 分区、共享内存、vring、carveout 的地址与大小要在 dts、u-boot、NuttX MMU 表、双侧头文件之间保持一致，任何一处不一致都会导致**静默失败**。
5. **单串口复用**：三个域只有一根物理调试串口可用。

3.1.3 创新点

- **技术层面**：三个域、两种指令集（AArch64 + ARMv6-M）、两个独立 NuttX 实例、共用一根调试串口；把 RK3588 一直闲置的 PMU Cortex-M0 点亮并跑上了 NuttX（官方 AMP 参考中此核为空白）；修复了 NuttX 通用层的两处缺陷和一处 GICv3 通用问题，而不只是加板级代码。
- **场景层面**：成品是一个带 NPU 实时人物识别、开机自启、上电即用的完整摄像头终端，具备可演示的产品形态。
- **交互层面**：两个 NuttX 的 nsh 都通过 rpmsg 隧道成 Linux 侧的 `/dev/ttyNSH0` / `/dev/ttyNSH1`，不占物理 UART，可随时交互调试从核。

3.2 系统方案设计

3.2.1 系统总体架构



数据流: Linux 侧 `amp_fb_show` 从 imx415 采集 → RGA 缩放/转换 → RKNN 推理得到检测框 → 写入共享内存的帧槽与检测结果块 (seqlock 保护) → 通过 rpsmsg 门铃通知 NuttX; NuttX(cpu_l3) 读共享内存、用 LVGL 合成界面与识别框, 双缓冲原地扫描到 VOP 硬件图层出画面。

3.2.2 方案论证与选型

开发板选型: RK3588 EVB7 LP4 V11 官方评估板。理由: SoC 同时具备 4×A55 + 4×A76 大小核、一颗独立 PMU Cortex-M0、成熟的 ISP/NPU/VOP, 是验证「片内 AMP 三域」这一命题的理想载体; 且带 MIPI-DSI 屏、imx415 摄像头、AP6398S WiFi/BT, 外设完整可做成品演示。

端/云职责划分 (含降级): 本作品推理全部在端侧 NPU 完成 (RKNN, yolov5s), 无云依赖, 天然抗断网/弱网。UI 与显示由 NuttX 独立承担, **Linux 崩溃时 NuttX 仍在出画面** (崩溃隔离); 显示图层从 VOP 硬件层直出, 不依赖 Linux 图形栈。

与备选方案对比：

方案	代价 / 结论
NuttX 独占摄像头硬件	需重写 rkCIF (约两万行)、ISP/3A 闭源 (librkaiq)、NPU 缺闭源编译器；且 vop_mmu 整个 VOP 共享、irq_vop 与 vop_mmu 共用 GIC_SPI156 无法切分 → 否决
双芯片 (MCU + AP)	成本/功耗/同步复杂度上升，且非"新硬件适配"命题 → 否决
片内 AMP，Linux 采集推理 + 共享内存 + rpmsg 通知 (选定)	复用 Linux 闭源采集/推理链，NuttX 只做显示与交互，大块数据走共享内存、rpmsg 只传通知，性价比最高

3.2.3 关键模块设计（各节点角色与运行系统）

节点	处理器	运行系统	角色
域 1	A55 × 7 (cpu0-2,4-7)	Linux (vendor kernel 6.1.141)	rpmsg master/host；采集、推理、WiFi/BT、桌面
域 2	A55 × 1 (cpu_l3 = cluster0 core3, MPIDR 0x300)	openvela / NuttX (AArch64)	rpmsg remote；LVGL UI + 触摸 + 直驱 VOP 图层
域 3	PMU Cortex-M0 (ARMv6-M)	openvela / NuttX	rpmsg remote；独立 RTOS 实例

通信 / 协同： 两条独立 virtio link 共用一个 mailbox (@0xfec60000) —— link1 连 cpu_l3↔Linux (virtio0)，link2 连 M0↔Linux (virtio1)。大块帧与检测结果走**共享内存**，控制/通知走 **rpmsg 门铃** (mailbox A2B 通道, INTID 97-100)。

控制台复用： 三域共用一根 UART2@1500000 8N1 (Linux console=ttyFIQ0)；NuttX 侧用非缓存轮询写打印 [AMP]/[M0] 状态行，两个 nsh 则通过自研的 rpmsg_nsh_tty 驱动隧道成 Linux 侧 /dev/ttyNSH0、/dev/ttyNSH1，不占物理串口。

3.3 核心算法与技术原理

3.3.1 AI 算法实现（端侧）

- 模型 / 框架：** yolov5s (输入 640×640)，RKNN 格式 (yolov5s-640-640.rknn, 8.3 MB)，运行时 librknnrt 2.3.2 + rknpu 驱动 0.9.8。
- 量化：** INT8 (RKNN 量化模型)，配合 COCO 80 类标签。
- 算力占用与逐核实测** (50 次采样、弃首帧)：

- 单核 (RKNN_NPU_CORE_0) : run **17.3–17.9 ms** (=hw 时间, 纯硬件、几乎零 CPU) , ≈ 55 fps。
- 三核 0+1+2: run 约 9.96 ms (约 $1.74\times$ 加速, 非 $3\times$, 收益递减) 。
- **决策**: 因目标帧率仅需 10–15 fps, 余量巨大, **固定单核**运行, 把另外两核 NPU 留给系统, 降功耗与争用。
- **端到端单次检测预算**: NPU 17.3 ms + CPU 1.6 ms (输入转换 + 输出取出 + 后处理 + 释放) = **18.9 ms $\rightarrow$ 约 52 fps 上限**; demo 实测 average run 18.92 ms。

3.3.2 关键机制设计

- **RGA 硬件加速的输入/输出转换**: 用 RGA 做缩放与格式转换, 建立成本模型 ≈ 1.7 ns/源像素 + 2.1 ns/目标像素; 引入 **dma-buf 零拷贝** (-z) 后, RGA 单次从 4466 μ s 降到 1516–2100 μ s (每帧省 2.3–2.9 ms) , 单帧 26.9 ms $\rightarrow$ 22.5 ms, CPU 占用 81% $\rightarrow$ 68%。
- **NPU 输出去交织**: 确认 rknn_outputs_get 慢 (3.4 ms) 的真因是硬件原生布局 **NC1HWC2** (16 通道对齐补白) 与声明的 NCHW 之间做跨步去交织, 用 RKNN_QUERY_NATIVE_OUTPUT_ATTR 坐实, 非线性拷贝可省。
- **检测质量过滤**: 厂商 postprocess 分别对 class 与 obj 两个因子各卡 0.25 阈值, 却从不卡二者乘积, 导致约 44% 的框是垃圾 (107 框中 47 个低于 25%)。改为按乘积分数过滤, 35% 阈值下留 47 框 (40 person / 7 其他) , 并把 pad 颜色对齐厂商 (114 $\rightarrow$ 128) 使 person 最高置信度从 69% 提升到 89%。
- **共享内存 seqlock**: 帧槽与检测结果块用奇偶序号 seqlock 无锁同步, 实测 1327 次读 torn 0 / bad 0。

3.3.3 openvela 系统能力的深度运用

本作品落地 openvela **图形、AI、多媒体**三项核心能力:

- **图形**: LVGL 应用 (主页 + 相机页) , NuttX 直接编程 VOP Esmart3 硬件图层 + /dev/fb0 双缓冲原地扫描 (无 memcpy、无撕裂) , goodix 触摸 /dev/input0。
- **多媒体 / AI 协同**: 摄像头帧与检测结果经共享内存跨域传递, /dev/amcam0 消费端。
- **系统 / 移植**: 新建两套 chip 层与 board 层、rpmsg 传输层、资源表; 用到 openvela 的 rptun、uart_rpmsg、arm64/armv6-m arch、arm_systick 等组件。

对 openvela 的优化 / 改进 (已回馈到通用层) :

1. arch/arm64/src/common/arm64_gicv3.c: 原按 up_cpu_index() 计算 redistributor 基址, AMP 下永远选中 cpu0 的 GICR, 从核 timer PPI 永不使能。改为按 **MPIDR affinity 匹配 GICR_TYPER 探测本核 rdist** (cpu_l3 $\rightarrow$ 0xfe6c0000) ——通用修复。
2. drivers/serial/uart_rpmsg.c: dmasend 在 rpmsg 未 ready 时阻塞等待会与对端互锁、首次 NS announce 必丢且不重试。修正为**丢弃首帧欢迎语并加 re-announce**。
3. arch/arm64/src/common/arm64_arch_timer.c: 改用 EL1 物理定时器 CNTP/PPI30, 避开 AMP 下 CNTVOFF 依赖。

改进建议：建议 openvela 上游补充 RK3588 AMP 从核（含 PMU M0）的参考 chip 层与 rpmsg 示例，并在文档中明确「GIC 由主核初始化、从核只探测自身 rdist」这一 AMP 约定。

3.4 系统实现

3.4.1 软件 / 固件架构

- **NuttX cpu_l3 (AArch64)**：新建 arch/arm64/src/rk3588/（serial / rptun / rsctable / boot(MMU) / Make.defs、include/rk3588/irq.h）+ board boards/arm64/rk3588/evb7-amp/（bringup / fb / touch / shm / vop / cam）。
- **NuttX M0 (ARMv6-M)**：新建 arch/arm/src/rk3588-m0/（含 hardware/rk3588m0_memorymap.h，把 TRM 地址重映射模型写成代码）+ board boards/arm/rk3588-m0/evb7-m0/。
- **Linux 例**：AMP dts、Mali G610 与 WiFi-BT 两个 config fragment、drivers/tty/rpmsg_nsh_tty.c，用户态主程序 rk3588-amp-demo/amp_fb_show.c（采集/RGA/RKNN/旋转/发布）。
- **u-boot**：configs/rk3588_defconfig 开启 CONFIG_AMP / CONFIG_ROCKCHIP_AMP，负责把两个固件从 FIT 拉起（A55 走 PSCI，M0 走 SIP + 解复位）。

3.4.2 数据流与关键流程

```
[imx415]→CSI→[rkisp mainpath 1920×1080 / selfpath 1280×720]
→ RGA 缩放/格式(dma-buf 零拷贝) → RKNN INT8 推理 → 后处理(乘积阈值 35%)
→ 写共享内存: 帧槽(2MB×N) + 检测块(seqlock) → rpmsg 门铃通知
[NuttX cpu_l3] 读共享内存 → LVGL 合成(界面+识别框) → 双缓冲 → 直写 VOP Esmart3 图层 → MIPI-DSI 屏
```

3.4.3 硬件设计与适配（新硬件适配赛道核心得分项）

是否完成全新硬件平台适配 / 驱动开发：是。

- **芯片 / 开发板**：Rockchip RK3588（8 核）/ EVB7 LP4 V11。
- **外接模块与接口**：MIPI-DSI 屏 1080×1920（旋转后 540×960）、goodix gt1x 电容触摸（实测量程 720×1280）、imx415 MIPI 摄像头（sensor 3864×2192，硬件缩放到 1280×720 / 512×288）、AP6398S（BCM4359）WiFi/BT（SDIO bcmdhd + /dev/ttyS9 HCI attach）。
- **关键 BOM**：RK3588 EVB7 V11 主板、MIPI-DSI 屏、imx415 模组、AP6398S 模组、eMMC。
- **适配类型与难点/解决：**

目标	此前状态	适配内容	关键难点与解决
RK3588	上游仅有	新建 rk3588 chip NR_IRQS 181→512；GICv3 rdist 按 MPIDR 探	

目标	此前状态	适配内容	关键难点与解决
A55 (cpu_l3, AArch64)	rk3399 arm64 chip 层	层 + evb7-amp board 层	测; EL1 物理定时器; VOP 图层直驱
RK3588 PMU Cortex-M0 (ARMv6-M)	官方 AMP 参考完全没有	新建 rk3588-m0 chip 层 + evb7-m0 board 层	由 TRM Table 9-3 推出 512MB 取指/外设重映射窗口 (64KB 粒度); FIT 加 uc_start/uc_end 让 u-boot 真正编程 CODE_START 并解复位; SysTick 复用 arm_systick

内存布局（一致性关键）：amp-core@0x30000000（16 MB）= cpu_l3 固件；amp-shmem@0x31000000（8 MB，MT_NORMAL_NC 非缓存）= rpmsg 共享内存 + 帧/检测块；vring0/1 固定 0x07c00000/0x07c08000、buffer pool 0x08000000（2 MB）；mcu_reserved@0x07a00000（2 MB）= M0 固件。分区表在 oem 后加 16 MB amp 分区（LBA 0x01CB8000）。

3.4.4 应用 / 交互端设计

LVGL 主页（带呼吸心跳点）→ 点相机图标进相机页 → 实时画面 + 人物识别框；触摸交互；开机自启，上电即用无需命令。

3.4.5 自定义 Skill（硬性要求：≥1，本作品沉淀 2 个）

沉淀于 logs/.kiro/skills/：

- 1. **board-change-tracker**：触发场景为在 Armbian/openvela 框架内做板级适配（新建 board conf、改 family、加 kernel/u-boot patch、改 defconfig、接 rkbin blob 等）。作用是每次改动立即向 per-board changelog 追加「What/Type/Change/Why」四要素条目，项目收尾时用 git status 交叉核对再一次性完整提交，确保**没有改动被遗漏或漏提交**。本作品 6700+ 行的主 changelog 即其产物。
- 2. **session-handoff**：触发场景为会话接近上下文上限或用户要求「交接/总结继续」。作用是把当前状态蒸馏进单一自包含交接文档（现状/源码树绝对路径与分支/已验证里程碑/进行中含未编译文件/关键技术要点/known-good 命令/TODO/提交哈希），新会话只读一份即可续上，并诚实区分 verified 与 unverified。

说明：本作品为「新硬件平台适配」赛道，Skill 用于**开发工作流**（AI 协作的工程日志与交接）。若面向 AI 硬件产品创新赛道的运行时 Skill (/data/agent/skills/) 要求，可在后续迭代中补充设备端触发的语音/视觉 Skill。

3.5 系统测试与结果分析

3.5.1 测试环境

- 硬件：RK3588 EVB7 V11, MIPI-DSI 屏、imx415、AP6398S；启动介质 eMMC。
- 串口：UART2 @ 1500000 8N1, picocom 观测；主机 Linux + rkdeveloptool/adb。
- 工具链（四套，不可混用）：系统 aarch64-linux-gnu-gcc（内核 + Linux 用户态）；linaro 6.3.1-2017.05（vendor u-boot）；aarch64-none-elf（NuttX cpu_l3）；arm-none-eabi（NuttX M0）。

3.5.2 功能测试

功能	判据	结果
AMP 三域生效	nproc=7; dmesg 两条 rpmsg host is online	通过
M0 活体	devmem 0x07ae0000=0x414D5030("AMP0"), counter 每秒 +1	递增（SysTick+调度在跑）
两个 nsh over rpmsg	/dev/ttyNSH0 uname=aarch64; /dev/ttyNSH1 uname=arm/evb7-m0	均可交互
NuttX 直驱显示	开机即出 LVGL 主页，点图标进相机页	出画面且带识别框
端到端识别	bus.jpg → 4 person + 1 bus，置信度 0.30–0.88	结果正确

3.5.3 性能测试（真机实测，列表）

指标	实测数据	条件 / 样本
NPU 单核推理	run 17.3–17.9 ms （≈55 fps）	50 次采样弃首帧，纯硬件时间
NPU 三核加速	run ≈9.96 ms（约 1.74×）	收益递减，故选单核
端到端单次检测	18.9 ms （≈52 fps 上限）	NPU 17.3 + CPU 1.6
RGA 转换（dma-buf 零拷贝）	1516–2100 μs（较 4466 μs 省 2.3–2.9 ms/帧）	1280×720→640×640
摄像头显示路径（向量化后）	合计 ≈ 4.0 ms/帧 （30 fps 仅占一个 A55 的 12%）	724 帧统计，convert 快 5.4×
双路 ISP	30.0 / 30.2 fps, gaps 0/0（零丢帧）	299/297 帧统计，第二路 +41.5 MB/s DDR
相机页整页帧率	均 47.2 ms（ 21.2 fps ），最好 25.9 / 最	71 个周期约 144 秒；瓶颈 LVGL

指标	实测数据	条件 / 样本
	差 16.4 fps	82%
真推理满载检测链	均值 28.1 ms（占 33.3 ms 帧预算 84%）	1470 次推理，5 s 节拍

3.5.4 可靠性与稳定性测试

- **识别质量**：乘积阈值 35% 过滤后 person 类占多数（旋转真机 183 次检出，person 为最多类）；根因排查发现一次识别率偏低实为**镜头虚焦**（dump 图梯度能量 0.92，正常 5–20，代码 diff 为空）——具备可诊断性。
- **漏检兜底**：中断化 rpmsg RX 之外保留 100 ms 兜底，防止门铃中断被静默丢失；共享内存存在性判据改为**单调递增计数器**（避免软复位后 DRAM 旧 magic 伪装成"在线"）。
- **连续运行**：真推理上板 44.2 秒 1327 frames (30.0 fps) | notify 1754 copied 1327 dropped 1 torn 1 bad 0，NPU 满载下两路 ISP 仍 gaps 0/0；torn 0 / bad 0 验证 seqlock。rpmsg 压测 rx_count 涨到 10000、**0 assert**。
- **异常恢复 / 一致性**：定位并修复 MMU 映射短于 carveout 导致的**约 1027 帧后越界断言**（ESR_EL1 0x96000006），修后 copied 撑到 1147+、dump_assert 0。
- **冷启动一致性**：两次独立冷启动顺序一致；日志中 153 处 cannot|failed|error 逐一核过——150 条内核常规 + 3 条 u-boot/SPL 既有噪声，**零应用层报错、零断言**。

3.6 AI-Native 开发说明

本作品 全程由 Kiro 辅助开发 ，完整对话日志见 logs/，工程日志见 logs/.kiro/board-changes/。	
指标	数据
代码占比	约 85%（估算） 。口径：两套 chip 层、两套 board 层、rpmsg 传输、LVGL 应用、Linux AI Coding 侧 amp_fb_show.c 的采集/RGA/RKNN/后处理主体由 AI 编写；人负责给约束、上板验证、判断哪些"看起来对"的结果不可信。改动横跨 6 个仓库（kernel / u-boot / nuttx / apps / rk3588-amp-demo / armbain），NuttX 侧约 73 个文件。
使用的 AI 工具	Kiro （Autopilot / 交互式协作，全程中文）
MCP 工具使用情况	本项目以 Kiro 内置工具链（文件读写、终端、代码检索、子代理）为主完成移植与调试；未接入外部 MCP 服务器。
Skills 使用与新增	新增沉淀 2 个自定义 Skill ：board-change-tracker（逐条工程日志 + 收尾交叉核对提交）、session-handoff（跨会话自包含交接）。二者贯穿全程，是"上下文可续"的关键。

指标	数据
Token 使用总量	未使用 MiMo 云端模型（推理在端侧 NPU），无 MiMo Token 计费；Kiro 会话 Token 用量未单独统计。

AI 对开发效率的提升（真实案例）：

- **读 vendor 代码成本大幅下降：** RK3588 BSP 内核 + vendor u-boot 是数十万行、近乎无文档。「mailbox 握手先写 DAT 再写 CMD」「rockchip Linux 从不读 remote 的 resource table，须自己钉 DRIVER_OK」「rpmsg_char 的 id_table 只认 rpmsg-raw，端点取此名即可零内核改动拿到 /dev/rpmsg0」等关键结论，均由 AI 读源码得出。
- **调试是 AI 帮助最大的环节：** M0 点亮卡三轮的真因（u-boot 缺 uc_start/uc_end 落进 __weak 空桩却照打 ...OK）、重连必崩的真因（rpmsg_nsh_tty_install() 漏 tty_port_get() 导致 kref 不平衡 UAF），都靠 AI 读上游源码对照定位。

遇到的问题与解决方式（沉淀为方法论）：

changelog 专门记录了「看起来像成功的返回值」骗人的反复教训，并转化为纪律：

1. mailbox 打印的 version 0x0100 是驱动常量（probe 只 ioremap 不碰硬件）；
2. open("/dev/ttyM0")→0 的真实含义是 fd 0/1/2 全空，而非"打开成功"；
3. u-boot 的 ...OK 来自 __weak 空桩；
4. NS 侧读 GICD_IGROUPR 恒为 0（RAZ），曾据此误推出"必须改 OP-TEE"并真的踩坑；
5. 软复位不清 DRAM，no-map 区旧 magic 会伪装成当前状态 → 判据改用单调递增计数器；
6. AMP_SHM_SIZE 写在三处（dts / MMU 表 / 头文件），三种失败方式不同 → 改尺寸先 grep 全部实例。

结论纪律：**先确认目标代码路径是否真被执行，再解释"执行结果为何不对"**；性能数字报 mean/max 并弃首帧；等价性测试抽真实源码用 200 种子并补角点方向用例（对称测试测不出符号错）。

3.7 总结与展望

3.7.1 成果总结

在单颗 RK3588 上跑通了 **openvela + Linux 三域 AMP**，点亮了官方 AMP 参考中空白的 PMU Cortex-M0，新建了两套 chip/board 层并修复了 NuttX 通用层三处缺陷，最终交付一个开机自启、端侧 NPU 实时人物识别的摄像头终端：相机页 21.2 fps、双路 ISP 零丢帧、连续运行零断言。

3.7.2 应用前景与商业价值

- **目标受众：** 智能家居/安防摄像头厂商、需要"确定性 UI + 强算力"的边缘 AI 终端团队（如工业

HMI、门禁、儿童陪伴设备）；面向对**成本、崩溃隔离、开机速度**敏感的中高端消费与商用市场。

- **商业模式**：以「片内 AMP 参考设计 + 移植服务」形式对外，帮助客户在**单芯片**上同时获得 RTOS 的确定性与 Linux 的生态，省去第二颗 MCU 的 BOM 与同步成本。
- **规模化潜力**：方案与 RK3588 生态强绑定，可复用到同系列 SoC 的大量已有开发板；两个 Skill 与工程日志方法论本身也可作为团队级的 AI 协作资产复用。

3.7.3 不足与未来工作

- 相机页整页帧率受 LVGL 软件合成限制（占 82%），30 fps 需将 LVGL 单帧从 37.8 ms 降到 24.5 ms，可探索硬件图层合成 / 局部刷新。
- PMU M0 目前跑心跳与验证性负载，尚未承担业务功能，可扩展为低功耗待机/传感器聚合域。
- 运行时（设备端）Skill 尚未落地，后续可补充设备端触发的语音/视觉交互 Skill。
- 镜头对焦等硬件因素对识别率影响较大，产品化需加入自动对焦/图像质量自检。

附：理解本作品最快的入口是 `logs/.kiro/board-changes/rk3588-evb7-v11.md`（6700+ 行主 changelog，逐里程碑记录改动、真机验证与每一次判断错误的根因）；复现步骤见 `logs/.kiro/board-changes/SETUP-rk3588-evb7-v11.md`。